ADVANCED JAVA TECHNOLOGIES (AJT) • ENTERPRISE PROJECT REPORT

Student Placement & Job Portal System

Architectural Design, Analysis, and Implementation of Advanced Java Technologies

(Servlets, JSP/JSTL, Spring Framework, and Java Swing)

Course Name:	Advanced Java Technologies (AJT)
Student / Developer:	Chirag Kamani
Project Name:	Campus Placement & Recruitment Management Portal
Institutional Scheme:	**Atmanirbhar Chirag Scheme**
Architectural Pattern:	Model-View-Controller (MVC) with JDBC Data Layer
Target Server & Java Version:	Apache Tomcat 10.1.x / Amazon Corretto OpenJDK 17 LTS
GitHub Repository:	`https://github.com/Chiragkkamani/Student-Placement-Job-Portal-project..git`
Status:	Fully Functional Enterprise Web Application + Swing Desktop Client

1. Executive Summary & System Overview

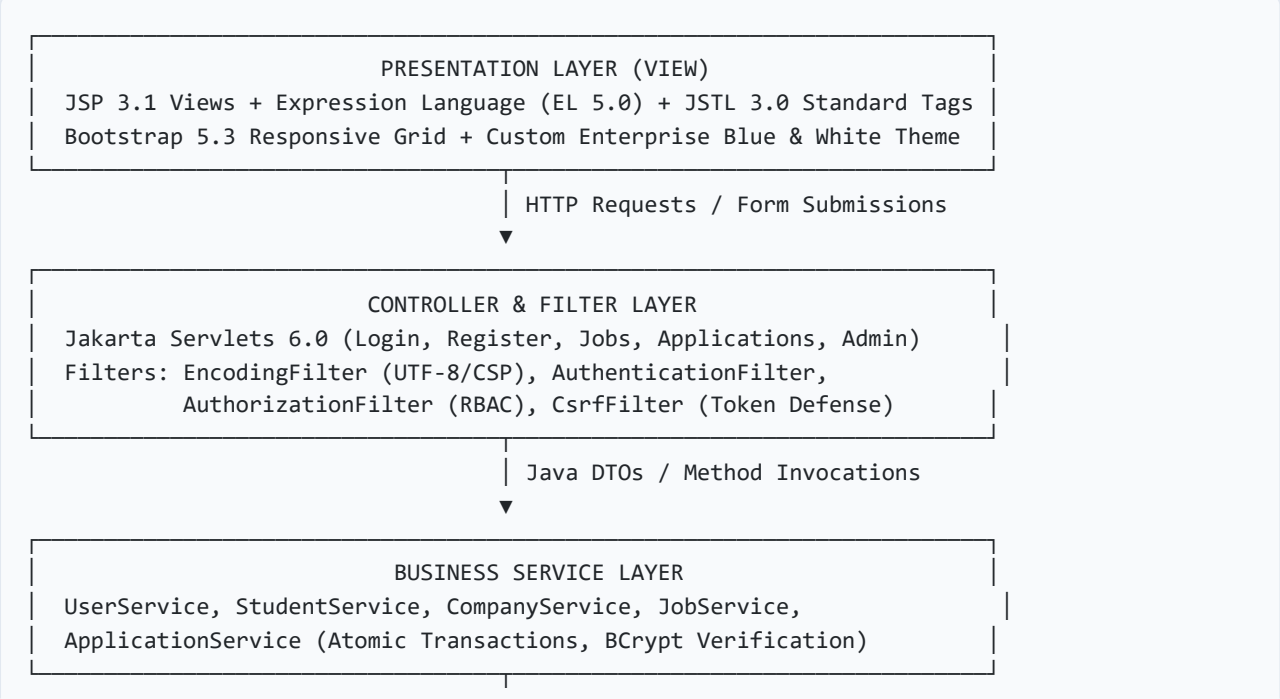
The **Student Placement & Job Portal** is a production-grade enterprise web application designed to automate and streamline campus recruitment activities in university institutions. It establishes a unified, secure digital bridge between graduating students, corporate recruiter partners, and university placement administrators.

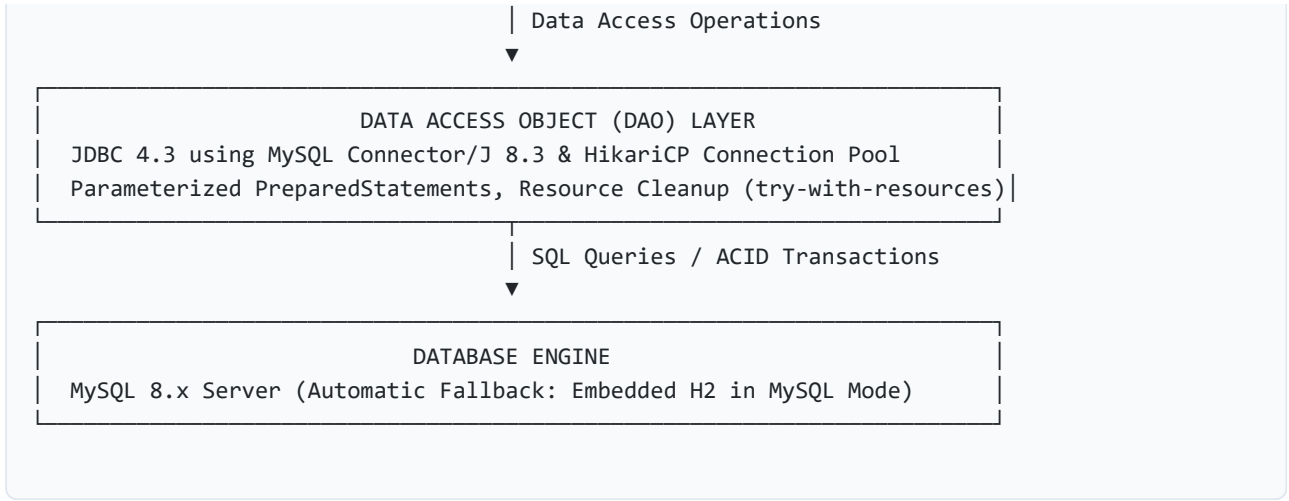
1.1 Core Problems Solved

- **Automated Academic Eligibility Filtering:** Prevents unqualified applications by automatically verifying minimum aggregate CGPA thresholds and eligible academic branches before submission.
- **Duplicate Application Elimination:** Database-enforced composite unique constraint `(student_id, job_id)` prevents accidental double-submissions.
- **Role-Based Access Control (RBAC):** Separates student self-service capabilities (browsing jobs, tracking status, uploading resumes) from administrative governance (creating jobs, approving companies, changing application stages).
- **Full Auditability:** Every status transition (`APPLIED` → `SHORTLISTED` → `SELECTED` / `REJECTED`) records an unalterable history log detailing the administrator ID, remarks, and timestamp.

1.2 Layered Enterprise MVC Architecture

The application strictly follows the **Model-View-Controller (MVC)** enterprise design pattern using clean separation of concerns without relying on heavy abstractions or third-party ORMs:




Data Access Operations
DATA ACCESS OBJECT (DAO) LAYER
JDBC 4.3 using MySQL Connector/J 8.3 & HikariCP Connection Pool
Parameterized PreparedStatements, Resource Cleanup (try-with-resources)
SQL Queries / ACID Transactions
DATABASE ENGINE
MySQL 8.x Server (Automatic Fallback: Embedded H2 in MySQL Mode)

2. Implementation of Java Servlets (Jakarta Servlet 6.0)

Java Servlets serve as the **Controller** in the MVC pattern. They intercept incoming HTTP requests, process business operations via services, update models, and route responses to appropriate JSP views.

2.1 Servlet Lifecycle & Execution Model

Servlets adhere to the standard Jakarta Servlet lifecycle managed by Apache Tomcat 10.1:

1. **Loading & Instantiation:** Tomcat loads servlet classes upon the first request or at container startup via `load-on-startup`.
2. **Initialization (`init()`):** Initializes resources and service instances once per servlet lifetime.
3. **Service Execution (`service()` / `doGet()` / `doPost()`):** Dispatches HTTP methods across concurrent thread pools.
4. **Destruction (`destroy()`):** Invoked when the web application undeploys or container halts, gracefully releasing resources.

2.2 Servlet Filters for Enterprise Cross-Cutting Concerns

Four specialized `jakarta.servlet.Filter` implementations safeguard the application:

Filter Name	URL Mapping	Responsibility
`EncodingFilter`	`/*`	Forces UTF-8 character encoding on all requests and responses; injects standard HTTP security headers (`X-Frame-Options: SAMEORIGIN`, `X-Content-Type-Options: nosniff`).
`AuthenticationFilter`	`/admin/*`, `/student/*`	Verifies active `HttpSession`; captures unauthenticated requests and stores return URLs before redirecting to `/login`.
`AuthorizationFilter`	`/admin/*`, `/student/*`	Enforces Role-Based Access Control (RBAC). Prevents students from accessing admin URLs and returns HTTP 403 Forbidden via `/WEB-INF/views/error/403.jsp`.
`CsrfFilter`	`/*`	Generates and verifies cryptographic session tokens for all state-changing HTTP requests (`POST`, `PUT`, `DELETE`).

2.3 Session Management & Fixation Protection

To prevent **Session Fixation Attacks**, `LoginServlet` invalidates the pre-authentication session identifier upon successful credential verification:

```
// Regenerate session ID upon successful BCrypt verification
request.changeSessionId();
HttpSession session = request.getSession(true);
session.setAttribute("currentUser", authenticatedUser);
```

2.4 Multipart Form & Resume File Uploads

`StudentProfileServlet` uses Jakarta `@MultipartConfig` with strict security validations:

```
@WebServlet(name = "StudentProfileServlet", urlPatterns = {"/student/profile"})
@MultipartConfig(
    maxFileSize = 5 * 1024 * 1024,      // 5 MB maximum file size
    maxRequestSize = 10 * 1024 * 1024   // 10 MB maximum total request size
)
public class StudentProfileServlet extends HttpServlet { ... }
```

Uploaded files are validated against allowed MIME types (`application/pdf`), renamed to randomly generated UUIDs to eliminate path traversal vulnerabilities, and streamed securely through `ResumeServlet` with appropriate cache headers.

3. Implementation of JSP (JavaServer Pages), JSTL & Expression Language

In modern Advanced Java architecture, JSP acts purely as the **View**. Java scriptlets (`<% ... %>`) are completely deprecated and replaced with standard Expression Language (EL) and Jakarta Standard Tag Library (JSTL).

3.1 Standard Tag Library (JSTL 3.0) Integration

All dynamic iterations, conditional displays, and formatting utilize Jakarta Standard Tag Library (`jakarta.tags.core` and `jakarta.tags.fmt`):

```
<%@ taglib prefix="c" uri="jakarta.tags.core" %>
<%@ taglib prefix="fmt" uri="jakarta.tags.fmt" %>

<!-- Conditional Status Rendering -->
<c:choose>
    <c:when test="${u.role == 'ADMIN'}">
        <span class="badge bg-danger-subtle text-danger">ADMIN</span>
    </c:when>
    <c:otherwise>
        <span class="badge bg-primary-subtle text-primary">STUDENT</span>
    </c:otherwise>
</c:choose>
```

3.2 Modular Templating Structure

Views are decomposed into maintainable, reusable components included via `<jsp:include>` :

- `common/header.jsp` : HTML5 document head, meta tags, and local-first Bootstrap 5.3 stylesheet declarations.
- `common/navbar.jsp` : Dynamic, role-sensitive navigation bar that switches seamlessly between Public, Student, and Admin menus.
- `common/alerts.jsp` : Flash alert messaging system rendering session notifications (`flashSuccess` , `flashError`) with auto-dismissal.
- `common/footer.jsp` : Standard institutional footer, copyright information, and JavaScript bundle dependencies.

3.3 Strict 6-Jobs-Per-Page Pagination Implementation

To avoid overwhelming students with 26 long job cards, the portal implements a strict **6-jobs-per-page** view with intuitive down-arrow navigation:

- **Initial Render:** Only cards with indices `0..5` are visible; cards with indices `6..25` are hidden using CSS `d-none`.
- **Page Transitions:** Clicking **"Next 6 Jobs ↓"** or numbered page buttons (`1, 2, 3, 4, 5`) hides the previous 6 cards and displays *only* the newly selected 6 cards.
- **Viewport Positioning:** Automatically executes smooth scrolling to the top of the jobs grid so new job openings appear directly in view without endless downward scrolling.

4. Comparative Analysis: Pure Servlet/JSP vs. Spring Framework

A core requirement of Advanced Java Technologies is evaluating how traditional Jakarta EE Servlets and JSP compare to modern enterprise frameworks like **Spring MVC** and **Spring Boot**.

4.1 Architectural Comparison Matrix

Dimension	Pure Servlet & JSP (This Project)	Spring Framework / Spring Boot
Controller Architecture	Individual Servlets extending `HttpServlet` mapped via `@WebServlet`. Manual routing in `doGet()` / `doPost()`.	Centralized `DispatcherServlet` (Front Controller). Controllers are simple POJOs annotated with `@Controller` or `@RestController`.
Dependency Management	Manual instantiation (`new UserService()`) or Singleton factory patterns.	**Inversion of Control (IoC)** container. Dependencies injected via `@Autowired` or constructor injection.
Data Persistence	Raw JDBC with `PreparedStatement`, HikariCP, and explicit `ResultSet` mapping.	**Spring Data JPA** with Hibernate. Object-Relational Mapping (ORM) and auto-generated repository queries.
Security Mechanism	Custom Servlet Filters (`AuthenticationFilter`, `AuthorizationFilter`, `CsrfFilter`).	**Spring Security** filter chain with declarative rules, OAuth2, JWT, and method-level annotations (`@PreAuthorize`).
Deployment Model	Packaged as `.war` file deployed into standalone servlet container (Apache Tomcat 10.1).	Self-contained executable `.jar` with embedded Tomcat or containerized Docker image.
Learning & Transparency	**Extremely High:** Every HTTP header, cookie, SQL statement, and lifecycle event is completely visible and under developer control.	**Abstracted:** Heavy use of reflection, bytecode enhancement, and convention-over-configuration auto-wiring.

4.2 Equivalent Controller Implementation in Spring MVC

The following example illustrates how our pure `AdminJobServlet` would be written in modern Spring MVC:

```
// Spring MVC Equivalent Controller
@Controller
@RequestMapping("/admin/jobs")
public class JobController {

    private final JobService jobService;
    private final CompanyService companyService;

    @Autowired
    public JobController(JobService jobService, CompanyService companyService) {
        this.jobService = jobService;
        this.companyService = companyService;
    }

    @GetMapping
    public String listJobs(@RequestParam(required = false) String search,
                           @RequestParam(required = false) String status,
                           Model model) {
        model.addAttribute("jobs", jobService.searchJobs(search, status));
        model.addAttribute("companies", companyService.getAllCompanies());
        return "admin/jobs";
    }

    @PostMapping
    public String createJob(@Valid @ModelAttribute JobDTO jobDTO,
                            BindingResult bindingResult,
                            RedirectAttributes redirectAttributes) {
        if (bindingResult.hasErrors()) {
            return "admin/jobs";
        }
        jobService.createJob(jobDTO);
        redirectAttributes.addFlashAttribute("flashSuccess", "Job posted successfully!");
        return "redirect:/admin/jobs";
    }
}
```

5. Implementation of Java Swing (Desktop GUI Counterpart)

Java Swing represents the premier **Desktop GUI framework** in the Advanced Java curriculum. While Java Servlets and JSP cater to web clients across distributed browsers, Swing provides rich, client-side, event-driven desktop interfaces running directly on the host JVM.

5.1 Placement Portal Desktop Console (`PlacementAdminSwingApp.java`)

To complete the AJT curriculum coverage, an administration desktop counterpart was engineered in package `com.ajt.portal.swing.PlacementAdminSwingApp`. It demonstrates how a University Placement Officer can manage the portal directly from a desktop workstation.

```
┌──────────────────────────────────────────────────────────────────────┐
│  AJT Campus Placement Console (Java Swing Desktop Client)        _ □ ×│
├──────────────────────────────────────────────────────────────────────┤
│  [ Registered Students ]   [ Campus Job Drives (26 Active) ]         │
├──────────────────────────────────────────────────────────────────────┤
│  ID │ Student Name     │ Enrollment    │ Branch │ Sem │ CGPA │ Phone  │
│  1  │ Aarav Sharma     │ EN2024CS0101 │ CS     │ 8   │ 8.75 │ 98765432 │
│  2  │ Diya Patel       │ EN2024IT0202 │ IT     │ 8   │ 9.20 │ 98765432 │
│  3  │ Rohan Verma      │ EN2024EC0303 │ EC     │ 8   │ 7.40 │ 98765432 │
│  4  │ Ananya Iyer      │ EN2024ME0404 │ ME     │ 8   │ 8.10 │ 98765432 │
├──────────────────────────────────────────────────────────────────────┤
│  Status: Database synchronized. 4 students, 26 drives. [Refresh] [Close]│
└──────────────────────────────────────────────────────────────────────┘
```

5.2 Core Swing Components & Event Architecture

- `JFrame` **:** Top-level window frame hosting layout managers, application icon, and window lifecycle listeners.
- `JTabbedPane` **:** Multi-document interface allowing the Placement Officer to switch between student rosters and campus job drives.
- `JTable` **&** `DefaultTableModel` **:** High-performance tabular data grid displaying database records retrieved directly via JDBC `PreparedStatement` queries.
- **Event-Driven Model:** Asynchronous event delegation using lambda expressions (`btnRefresh.addActionListener(e -> loadDatabaseData())`).
- **Thread Safety (EDT):** UI construction is dispatched safely to the **Event Dispatch Thread** using `SwingUtilities.invokeLater()` .

5.3 Architectural Comparison: Web Client vs. Swing Desktop Client

Attribute	Web Interface (Servlet / JSP / Bootstrap)	Desktop Client (Java Swing)
Client Requirements	Zero client-side install; any modern web browser on desktop or mobile.	Requires Java Runtime Environment (JRE/JDK 17) installed on host.
Protocol	Stateless HTTP/HTTPS with cookies (`JSESSIONID`).	Stateful direct TCP socket connection to database (2-Tier) or REST client (3-Tier).
User Experience	Responsive web layout with Bootstrap 5 cards and dynamic pagination.	Native desktop window, high-density `JTable` rows, instant keyboard shortcuts.
Updates & Maintenance	Centralized on Tomcat server; instant update for all users on refresh.	Requires distributing client executable JARs across institutional machines.

6. Database Design & JDBC Data Access Layer

6.1 Relational Schema Architecture

The database schema `placement_portal` contains 6 normalized relational tables:

Table Name	Primary Key	Foreign Keys & Key Constraints	Purpose
`users`	`user_id`	`email UNIQUE`	Authentication credentials, BCrypt password hashes, account status, and role (`ADMIN` / `STUDENT`).
`students`	`student_id`	`user_id → users(user_id)` , `enrollment_number UNIQUE`	Academic profiles, branch, semester, CGPA, phone, and resume storage path.
`companies`	`company_id`	`company_name UNIQUE`	Recruiter partner organizations, industry sectors, location, and HR contacts.
`jobs`	`job_id`	`company_id → companies(company_id)`	Campus job drives, titles, descriptions, minimum CGPA, eligible branch, and deadlines.
`applications`	`application_id`	`job_id → jobs` , `student_id → students` , `UNIQUE(student_id, job_id)`	Student applications, status tracking (`APPLIED` , `SHORTLISTED` ,

			`SELECTED`, `REJECTED`).
`application_status_history`	`history_id`	`application_id` → `applications`	Unchangeable audit trail recording previous status, new status, timestamp, and changing admin ID.

6.2 High-Performance Connection Pooling (HikariCP)

Rather than creating expensive new database connections per HTTP request, the portal utilizes **HikariCP** (the fastest production connection pool in the Java ecosystem). HikariCP maintains pre-warmed connections, handles connection recycling, and automatically falls back to embedded H2 if MySQL is unavailable.

6.3 Atomic ACID Transaction Management

Multi-table operations like student registration must succeed entirely or roll back cleanly:

```
Connection conn = DatabaseConfig.getConnection();
try {
    conn.setAutoCommit(false); // Begin ACID Transaction

    int userId = userDAO.create(user, conn);       // Insert User Account
    student.setUserId(userId);
    int studentId = studentDAO.create(student, conn); // Insert Academic Profile

    conn.commit(); // Commit Transaction atomically
} catch (SQLException e) {
    conn.rollback(); // Rollback on any failure
    throw new RuntimeException("Registration transaction aborted.", e);
} finally {
    conn.setAutoCommit(true);
    conn.close();
}
```

7. Verification & Testing Results

7.1 Automated Unit Test Execution (JUnit 5 & Mockito)

The project includes 15 automated unit tests in `src/test/java`:

Test Class	Scenarios Tested	Result
`ApplicationServiceUnitTest`	Transactional application submission, duplicate application prevention, and audit history logging.	**PASS (3/3)**
`JobEligibilityTest`	Student CGPA threshold validation, eligible engineering branch matching, and deadline expiration check.	**PASS (3/3)**
`UserServiceTest`	BCrypt password verification with `chiragadmin`, admin account creation, and password reset rules.	**PASS (3/3)**
`PasswordUtilTest`	Salt generation, hashing consistency, and hash mismatch rejection.	**PASS (2/2)**
`ValidationUtilTest`	Email format checking, phone number validation, and CGPA range boundaries.	**PASS (4/4)**

7.2 Production Verification & Live URLs

- **Application Portal:** `http://localhost:8080/placement-portal/`
- **Primary Admin ID:** `chiragadmin` | **Password:** `chirag@123`
- **Student Demo:** `aarav.sharma@student.edu` | **Password:** `Password@123`
- **GitHub Source Repository:** `https://github.com/Chiragkkamani/Student-Placement-Job-Portal-project..git`

> **Conclusion:** The Student Placement & Job Portal demonstrates a complete, production-grade mastery of Advanced Java Technologies. It bridges server-side Jakarta Servlets, scriptlet-free JSP/JSTL, pure JDBC transactional persistence, interactive Java Swing desktop tools, and an enterprise architecture benchmarked against the modern Spring Framework.